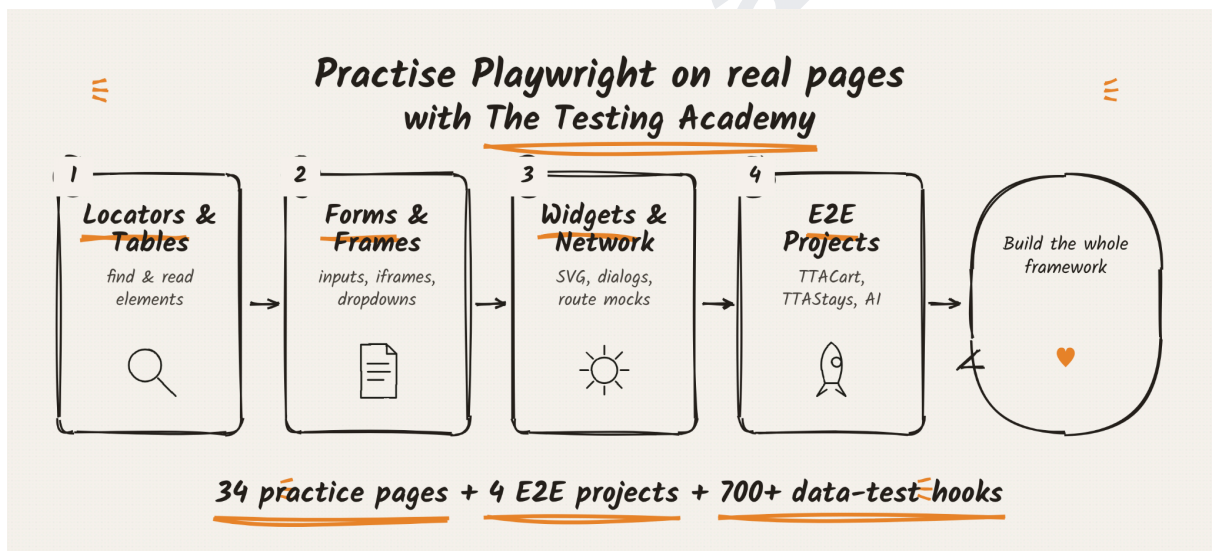




Playwright E2E: Full Guide with Advanced Framework

1. **Part 1:** Builds Playwright fundamentals lab by lab;
2. **Part 2 :** Grows them into a production framework with POM fixtures, API testing, BDD, reporting
3. **Part 3 :** AI Agent Factory + NPM registry, Docker Shard



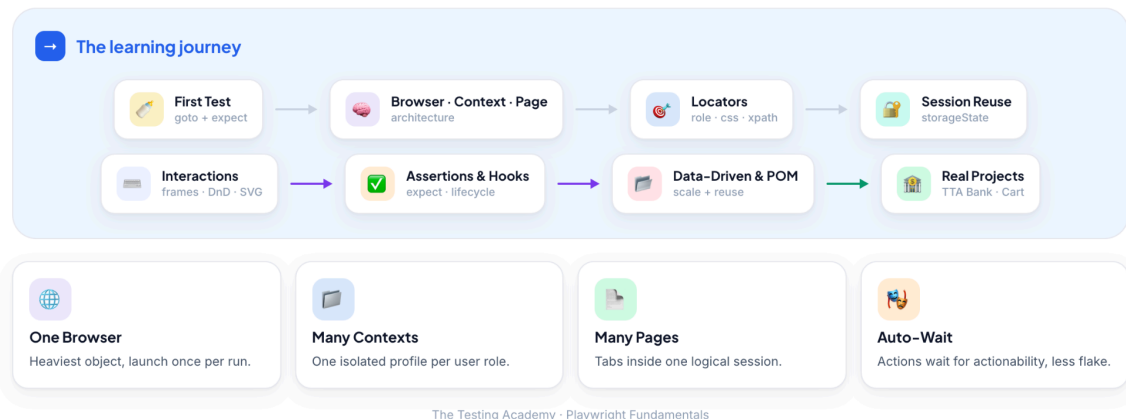
Part 1 - Playwright Fundamentals to Advance

From `npx playwright test` to a multi-context, session-cached, Allure-reported, CI-running suite, one topic at a time. Written for The Testing Academy Playwright + TypeScript course.



Playwright Maturity Roadmap

From your first `npx playwright test` to a full framework, one stage at a time



What is Playwright?

Playwright is a modern end-to-end testing framework from Microsoft that drives a real browser (Chromium, Firefox, or WebKit) to open pages, click, type, and assert, exactly like a real user. It is written in TypeScript, ships with auto-waiting, and gives you traces, videos, and screenshots out of the box.

A Playwright project has a standard shape:

- `tests/` holds your `.spec.ts` files
- `playwright.config.ts` holds the settings (browsers, reporters, retries, trace)
- `npx playwright test` runs the suite
- `npx playwright show-report` opens the HTML report

✅ **Is Playwright just Selenium 2.0?** No. Playwright talks to the browser over the DevTools protocol (not WebDriver), auto-waits for elements to be actionable, isolates every test in a fresh browser context, and bundles reporting and tracing. Less flake, less boilerplate.

Your First Test (Lab 209)

Every test file imports `test` and `expect`, then describes each test with `test('name', async ({ page }) => { ... })`. The `page` is a browser tab Playwright hands you.

```
test('has title', async ({ page }) => {  
  
  await page.goto('https://playwright.dev/');  
  
  await expect(page).toHaveTitle(/Playwright/);  
})
```



```
});
```

- `test(name, callback)` defines one test, the arrow function is the callback
- `async / await`: every Playwright action returns a promise, so you `await` it
- `page.goto(url)` opens a URL
- `expect(page).toHaveTitle(/Playwright/)` asserts the title matches a regex

■ Test Annotations (Lab 210)

Annotations change how a test runs without deleting it.

```
test.skip(browserName === 'firefox', 'Feature not yet supported on Firefox');
```

```
test.slow(); // triples the default timeout
```

```
test.fail(); // this test is expected to fail
```

```
test.only(); // run only this test (local debug, never commit)
```

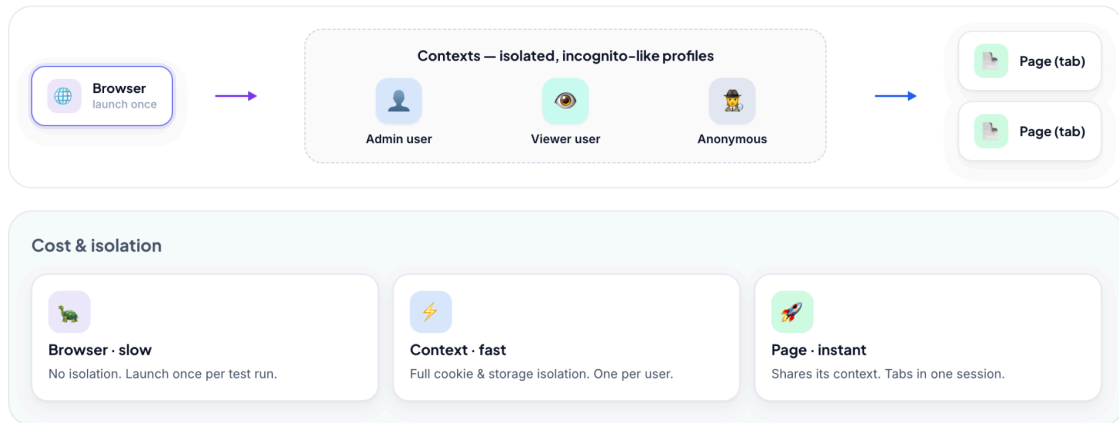
Annotation	Effect	Use For
<code>test.skip(cond, reason)</code>	skips when the condition is true	per-browser / per-env gating
<code>test.slow(cond, reason)</code>	triples the timeout	heavy pages
<code>test.fixme(cond?)</code>	marks known-broken, will not run	triage queue
<code>test.fail()</code>	test is expected to fail	TDD red phase
<code>test.only</code>	run only marked tests	local debug only

■ Browser, Context, and Page (Labs 211-218)

This is the heart of Playwright's architecture. The golden rule: **one Browser, many Contexts, many Pages**. A **Context** is an isolated, incognito-like profile with its own cookies and `localStorage`, so it is perfect for modelling one user.

Browser · Context · Page

Playwright's golden rule: one Browser, many isolated Contexts, many Pages



The Testing Academy · Browser / Context / Page

Object	Cost	Isolation	Use
Browser	slow	none	launch once per run
Context	fast	full (cookies / storage)	one per user role or per test
Page	instant	shares its context	tabs inside one logical session

There are two ways to get a page. The **manual** API launches all three levels yourself, and the **fixture** way (`{ page }`) lets Playwright create them per test.

// Manual 3-level launch (Lab 212) - you clean up in reverse

```
const browser = await chromium.launch();

const context = await browser.newContext();

const page = await context.newPage();

await page.goto('https://playwright.dev/');

await page.close();

await context.close();

await browser.close();
```



```
// Two isolated users in parallel (Lab 213)
```

```
const adminContext = await browser.newContext();
```

```
const viewerContext = await browser.newContext();
```

```
const adminPage = await adminContext.newPage();
```

```
const viewerPage = await viewerContext.newPage();
```

```
// adminContext cookies never leak into viewerContext
```

`browser.newContext({ viewport, locale, extraHTTPHeaders })` customizes a context (Labs 216-217), and `test.use({ viewport, locale })` applies config to every test in a `describe` block (Lab 218).

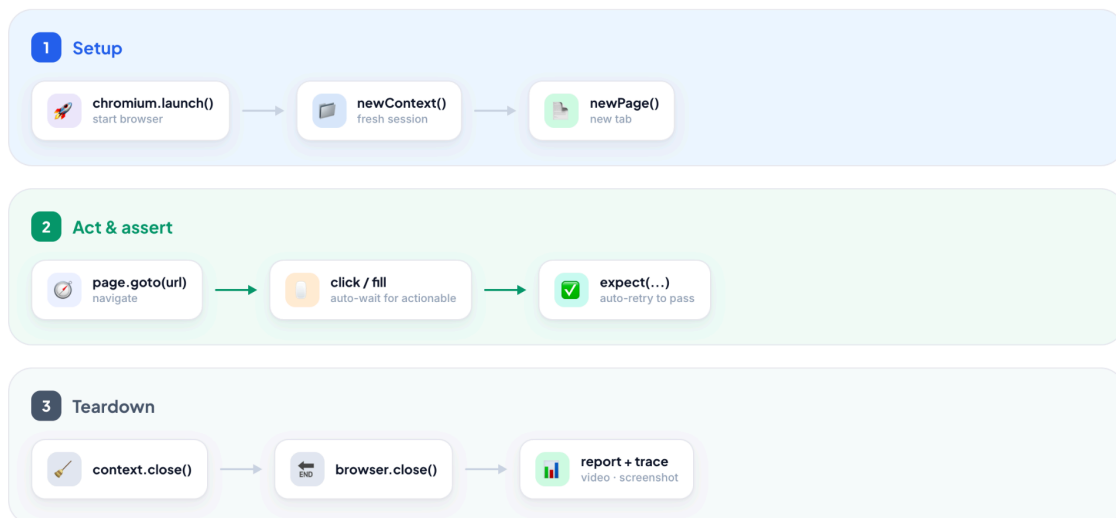
In Simple Terms

- Launch the Browser once, it is the heavy object
- Give each user their own Context (isolated cookies and storage)
- Open Pages (tabs) inside a context
- The `{ page }` fixture does all three for you, per test

How a Playwright Test Actually Runs

How a Playwright Test **Runs**

Launch, isolate, navigate, act, assert, close, with auto-waiting baked in



The Testing Academy · Test Lifecycle

The runner launches the browser, opens a fresh context and page, navigates, then each action auto-waits for the element to be actionable before running. Assertions auto-retry until they pass or time out, then the context and browser close.

Locators and Commands (Labs 219-227)

A **locator** is how you find an element. Locators have three defining properties.

- **Lazy:** creating a locator does nothing, it resolves only when you act on it
- **Auto-waiting:** it waits for the element to be visible, enabled, and stable (default 30s)
- **Strict:** it must match exactly one element, more than one throws a `StrictModeViolation`

// The locator family

`page.locator('#login-username');` // CSS id (Lab 222)

`page.locator('xpath=//div[@class="x"]');` // XPath (Lab 223)

`page.getByRole('link', { name: 'Make Appointment' });` // role, recommended (Lab 224)

`page.locator('a.list-group-item').first();` // index: first/nth/last (Lab 225)



```
await field.pressSequentially('hi', { delay: 200 }); // realistic typing (Lab 226)
```

page.goto() Wait Strategies (Lab 219)

```
await page.goto(url, { waitUntil: 'domcontentloaded' });
```

Option	Fires When	Speed	Use For
<code>commit</code>	server returned headers	fastest	API-only checks
<code>domcontentloaded</code>	HTML parsed	fast	SPAs that hydrate later
<code>load</code> (default)	all sub-resources loaded	medium	most pages
<code>networkidle</code>	no requests for 500ms	slowest	heavy AJAX (use sparingly)

Cookies (Lab 227)

```
const cookies = await context.cookies();
```

```
await context.addCookies([{ name: 'token', value: 'abc', url: 'https://app.vwo.com' }]);
```



Locator Strategy Cheat Sheet

Pick the most resilient locator that fits. User-facing locators survive design changes, XPath survives almost nothing.

Locator Strategy Cheat Sheet

Pick the most resilient locator that fits, user-facing first, XPath last

✓ Preferred — survives redesigns



getByRole

Accessibility-first. Survives DOM refactors and design changes.



getByLabel / Placeholder

Form fields. Survives class renames.



getByText

Visible text. Survives layout changes.

⚠ Use with care



getByTestId

Stable data-test hooks. Immune to most UI changes.



CSS #id / .class

Fine for stable ids. Breaks on visual refactors.

✗ Last resort



XPath

Powerful for complex DOM traversal, but brittle, resilient to almost nothing. Needed for SVG namespaces via `name()`.

The Testing Academy · Locators

Priority	Locator	Resilient to	Demoed in
1	<code>getByRole</code>	DOM refactors, design changes	Lab 224
2	<code>getByLabel / getByPlaceholder</code>	class renames	Lab 231
3	<code>getByText</code>	layout changes	Lab 231
4	<code>getByTestId</code>	all UI changes	TTA Bank
5	<code>CSS #id / .class</code>	visual changes only	Lab 222
6	XPath	almost nothing	Lab 223



■ Session Storage: Log In Once (Labs 228-229)

Why log in fifty times when you can log in once? Save the authenticated state to a JSON file, then reuse it in every test.

```
// Lab 228 - perform login, then save the session
```

```
await page.goto('https://app.vwo.com/#/login');
```

```
// ... fill credentials + click login ...
```

```
await context.storageState({ path: './user-session.json' });
```

```
// Lab 229 - reuse the saved session, skip the login entirely
```

```
test.use({ storageState: './user-session.json' });
```

```
test('goes straight to dashboard', async ({ page }) => {
```

```
    await page.goto('https://app.vwo.com/#/dashboard'); // already logged in
```

```
});
```

In Simple Terms: `storageState` snapshots cookies + localStorage to disk. `test.use({ storageState })` restores it, so the test starts already authenticated.

■ Allure Reporting (Lab 230)

Allure produces rich, hierarchical reports. You tag tests with epic → feature → story so results group the way the business thinks.

```
await allure.epic("VWO Login Tests");
```

```
await allure.feature("Essential features");
```

```
await allure.story("Authentication");
```

```
await allure.description("Verify that the login page works");
```

```
npx allure generate ./allure-results --clean -o ./allure-report
```

```
npx allure open ./allure-report
```

■ Multiple Elements (Lab 231)

When a locator matches many elements, use the plural methods to read them all at once.



```
const links = page.locator('a.list-group-item');

const texts: string[] = await links.allInnerTexts(); // all visible text at once

const all = await links.all(); // Locator[]

const count = await links.count();

for (const link of all) {

    console.log(await link.getAttribute('href'));

}
```

Method	Returns	Use Case
<code>.count()</code>	<code>number</code>	how many matches
<code>.first() / .last() / .nth(i)</code>	<code>Locator</code>	pick one
<code>.all()</code>	<code>Locator[]</code>	iterate with <code>for...of</code>
<code>.allInnerTexts()</code>	<code>string[]</code>	all visible text in one shot
<code>.allTextContents()</code>	<code>string[]</code>	includes hidden text

■ Web Tables (Labs 232-234)

Tables come in two flavours: static and dynamic. Prefer Playwright's native `hasText` filter over brittle XPath templates.

// Static (Lab 232): XPath template vs native filter

```
const path = `//table[@id="customers"]/tbody/tr[${i}]/td[${j}]`; // old school

const row = page.locator('#customers tbody tr', { hasText: 'Helen Bennett' }); // native

const country = await row.locator('td').nth(2).innerText();
```

// Dynamic (Lab 233): count rows, then loop and read each cell

```
const rows = page.locator('table[summary="Sample Table"] tbody tr');

const rowCount = await rows.count();

for (let i = 1; i <= rowCount; i++) {
```



```
const rowData = await rows.nth(i).locator('td').allInnerTexts();

console.log(`Row ${i + 1}:`, rowData);

}
```

■ Dropdowns, Selects, and Frames (Labs 234-238)

Real apps rarely use a plain `<select>`. There are three flavours of "dropdown", each with its own pattern.

// Native `<select>` (Lab 235)

```
await page.selectOption('#country', 'India');
```

// Custom div-based dropdown (Lab 236)

```
await page.locator('.dropdown-trigger').click();
```

```
await page.getByText('India', { exact: true }).click();
```

// React-Select, searchable (Labs 237-238)

```
await page.getByTestId('rs-single').click();
```

```
await page.getByTestId('rs-single-input').fill('play');
```

```
await page.getByRole('option', { name: 'Playwright' }).click();
```

Widget	Detect	Use
<code><select></code>	a real <code><select></code> tag	<code>page.selectOption(selector, value)</code>
Custom CSS dropdown	click reveals <code><div role="listbox"></code>	click trigger then <code>getByText(option, { exact: true })</code>
React-Select / Combobox	<code>role="combobox" + role="option"</code>	click then <code>fill</code> then <code>getByRole('option', { name })</code>



■ Frames and Iframes (Labs 239-241)

`frameLocator` is Playwright's window into `<iframe>` content. Treat it like a mini-page and chain `.locator()` calls off it. For nested iframes, chain multiple `frameLocator` calls.

```
const carFrame = page.frameLocator('#frame-one');

await carFrame.locator('#RESULT_TextField-1').fill('Hyundai i10');

await carFrame.getByText('Submit registration', { exact: true }).click();

// Nested iframe within iframe (Lab 241)

page.frameLocator('#pact1').frameLocator('#pact2').frameLocator('#pact3').locator('#glaf').fill('hi');
```

■ Keyboard, Hover, Drag & Drop, Right Click (Labs 242-247)

Low-level input APIs for things `click()` and `fill()` cannot express.

Lab	API	Scenario
242	<code>page.keyboard.press / down / up</code>	single keys, modifiers (<code>Shift+0</code>), arrows
244	<code>locator.hover()</code>	reveal hidden submenus
245	<code>source.dragTo(target)</code>	classic drag and drop
246	<code>mouse.move / down / up</code>	manual DnD for libraries that ignore <code>dragTo</code>
247	<code>click({ button: 'right' })</code>	context menu

// Lab 246 - manual mouse path for finicky drag-and-drop libraries

```
const sBox = (await source.boundingBox())!;

const tBox = (await target.boundingBox())!;

await page.mouse.move(sBox.x + sBox.width / 2, sBox.y + sBox.height / 2);

await page.mouse.down();

await page.mouse.move(tBox.x + tBox.width / 2, tBox.y + tBox.height / 2, { steps: 10 });
```



```
await page.mouse.up();
```

Rule of thumb: try `source.dragTo(target)` first, fall back to the manual `mouse` path only when the DnD library swallows it.

■ JS Alerts, Confirms, Prompts (Lab 243)

Native browser dialogs block the page, so you cannot click them with a locator. Register a one-shot `dialog` handler **before** the action that triggers it.

```
test('JS Confirm accept', async ({ page }) => {  
  
  page.once('dialog', async dialog => {  
  
    expect(dialog.type()).toBe('confirm');  
  
    expect(dialog.message()).toBe('I am a JS Confirm');  
  
    await dialog.accept(); // or dialog.dismiss()  
  
  });  
  
  await page.locator('button', { hasText: 'Click for JS Confirm' }).click();  
  
  await expect(page.locator('#result')).toHaveText('You clicked: Ok');  
  
});
```

Dialog Type	API	Notes
alert	<code>dialog.accept()</code>	no return value
confirm	<code>accept()</code> / <code>dismiss()</code>	maps to Ok / Cancel
prompt	<code>accept('text')</code> / <code>dismiss()</code>	pass text into <code>accept</code>

Use `page.once` (not `page.on`), or the handler stays alive across tests and swallows future dialogs.

■ Handling SVG Elements (Labs 248-250)

SVG nodes live in their own XML namespace. CSS selectors and `getByRole` work out of the box, but plain XPath tags fail: `//path` matches nothing. Use `name()` or `local-name()` on the wildcard `*`.



```
const circle = page.locator('#circle-blue');

await circle.click();

const bars = await page.locator('.bar').all();

for (const bar of bars) {

    const quarter = await bar.getAttribute('data-quarter');

    await bar.click();

}

// Namespace-safe XPath for SVG (Lab 250)

const states = await page

    .locator(`//*[name()='svg']//*[name()='text' and contains(@class,'sm_label')]`)

    .allTextContents();
```

Selector Style	SVG-safe?	Example
CSS <code>#id</code> / <code>.class</code>	yes	<code>page.locator('#circle-blue')</code>
<code>getByRole</code>	yes (if role present)	<code>page.getByRole('button', { name: /Q3 bar/ })</code>
<code>[data-*) attr</code>	yes	<code>page.locator('[data-quarter="Q3"]')</code>
XPath plain tag <code>//path</code>	no	use <code>//*[name()='path']</code> instead

Shadow DOM (Lab 251)

Web Components hide their internals behind a shadow root. Playwright pierces **open** shadow roots automatically: `getByTestId`, `getByRole`, and CSS locators all see through, even nested ones.

```
const card = page.getByTestId('card-account');
```



```
await card.locator('input[name="email"]').fill('student@thetestingacademy.com');

await card.getByTestId('card-account-submit').click();

// Nested shadow host - Playwright still pierces

await page.getByTestId('card-inside-email').fill('pramod@thetestingacdemy.com');

await page.getByTestId('card-inside-submit').click();
```

Open shadow roots are pierceable, closed ones are not. XPath does **not** pierce shadow boundaries, so stick to CSS / role / testId.

■ File Upload (Labs 252-253)

`setInputFiles` is the one true upload API. Point it at a real file on disk, or synthesize one in-memory with a `Buffer`.

```
// Single file from disk (Lab 252)

const filePath = path.join(__dirname, 'testdata.txt');

await page.locator('#file-upload').setInputFiles([filePath]);

await page.getByRole('button', { name: 'Upload' }).click();

await expect(page.locator('#uploaded-files')).toContainText('testdata.txt');

// Multiple files synthesized in memory (Lab 253)

await page.locator('div.pf-v6-c-multiple-file-upload input').setInputFiles([

  { name: 'file1.jpg', mimeType: 'image/jpeg', buffer: Buffer.from('...') },

  { name: 'file2.jpg', mimeType: 'image/jpeg', buffer: Buffer.from('...') },

]);
```

The input can be hidden (`display: none`), `setInputFiles` still works with no scroll, click, or `force: true`.

■ File Download (Lab 254)

A download is an **event**, not a return value. Race a `waitForEvent('download')` against the click using `Promise.all`, then persist with `saveAs`.



```
test('download file via waitForEvent', async ({ page }) => {

  await
  page.goto('https://app.thetestingacademy.com/playwright/widgets/upload-download');

  const [download] = await Promise.all([

    page.waitForEvent('download'),

    page.getByTestId('download-static').click(),

  ]);

  await download.saveAs('out/' + download.suggestedFilename());

});
```

Without `Promise.all` the listener can be registered after the event fires, and the test hangs until timeout.

■ Scroll to Element (Lab 255)

Playwright auto-scrolls before any action, but sometimes you need to scroll without clicking, for example to trigger lazy loading.

```
// Bring a specific element into view (idempotent, skips if already visible)

await page.getByTestId('section-lazy').scrollIntoViewIfNeeded();

// Window-level scroll when there is no anchor element

await page.evaluate(() => window.scrollBy(0, 1000));

await page.evaluate(() => window.scrollTo(0, document.body.scrollHeight));

// Assert lazy items loaded by polling the count

await expect.poll(async () => list.count(), { timeout: 10_000 }).toBeGreaterThan(initialCount);
```

■ Expect Assertions (Labs 256-257)

`expect()` is Playwright's assertion API with three families. **Locator / Page / APIResponse** assertions auto-retry until they pass or time out (you must `await` them). **Value** assertions run once and mirror Jest exactly.

```
// Value assertions - synchronous, no await
```



```
expect(1 + 2).toBe(3);

expect({ age: 20, role: 'admin' }).toEqual({ role: 'admin', age: 20 });

// Locator assertions - auto-retrying, await required

await expect(heading).toBeVisible();

await expect(heading).toContainText('filter', { timeout: 10_000 });

await expect(page.locator('footer a')).toHaveCount(16);

// Soft assertions - collect failures, keep running

await expect.soft(firstName).toBeVisible();

await expect.soft(firstName).toHaveValue("");

// Negation and page assertions

await expect(page.locator('#error')).not.toBeVisible();

await
expect(page).toHaveURL('https://app.thetestingacademy.com/playwright/widgets/calendar');
```

Family	Auto-retry?	Examples
Locator	yes	<code>toBeVisible</code> , <code>toHaveText</code> , <code>toHaveCount</code> , <code>toBeChecked</code>
Page	yes	<code>toHaveTitle</code> , <code>toHaveURL</code> , <code>toHaveScreenshot</code>
APIResponse	yes	<code>toBeOK</code>
Value (Jest-style)	no	<code>toBe</code> , <code>toEqual</code> , <code>toContain</code> , <code>toThrow</code>

Default auto-retry timeout is 5s. Override per call with `{ timeout: 10_000 }` or globally with `expect.configure({ timeout: 30_000 })`. Use `expect.soft` when checking several independent fields so one run reports them all.



■ Test Hooks and Grouping (Labs 258-261)

Real suites need shared setup, named steps, and occasionally an ordered scenario.

```
test.beforeAll(async () => { /* once per worker per file */ });

test.beforeEach(async ({ page }) => { await page.goto(URL); });

test.afterEach(async ({ page }, testInfo) => {

  if (testInfo.status !== testInfo.expectedStatus) {

    await page.screenshot({ path: `out/fail-${testInfo.title}.png`, fullPage: true });

  }

});

test.afterAll(async () => { /* tear down */ });

// Named steps show up in the report (Lab 259)

await test.step('open practice page', async () => { await page.goto(URL); });

// Ordered suite that shares state (Lab 261)

test.describe.serial('Checkout suite - must run in order', () => {

  test('open landing', async () => { /* ... */ });

  test('add to cart', async () => { /* ... */ });

});
```

`beforeAll` runs once per worker per file (not truly once with `fullyParallel: true`, use `globalSetup` for that). `describe.serial` cascades failures, so use it only for state-coupled flows.

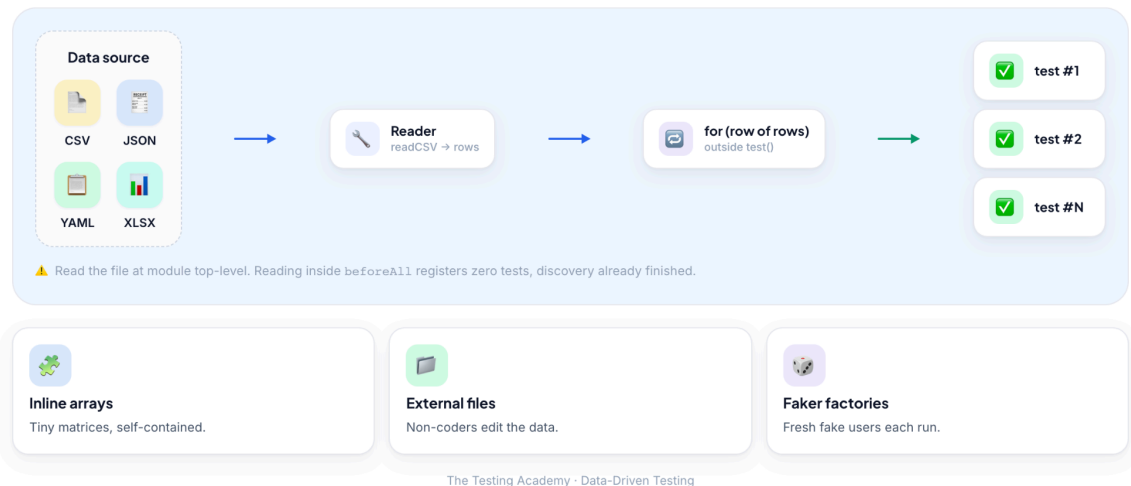
■ Data-Driven Testing (Labs 262-269)

Data-Driven Testing parameterizes one test body and feeds it many rows, so a single `test()` becomes N tests. The trick: wrap `test()` in a plain `for` loop **outside** the test, so N independent tests register at file-load time.



Data-Driven Testing

One test body + many rows = N tests. The loop lives outside `test()`



The Testing Academy · Data-Driven Testing

```
import { test, expect } from '@playwright/test';

import path from 'path';

import { readCSV } from './csvReader';

const loginData = readCSV(path.join(__dirname, 'login-data.csv'));

for (const data of loginData) {

  test(`Login with : ${data.description}`, async ({ page }) => {

    await
page.goto('https://app.thetestingacademy.com/playwright/multiple_element_filter');

    await page.getByRole('textbox', { name: 'Email Address' }).fill(data.username);

    await page.getByRole('textbox', { name: 'Password' }).fill(data.password);

    await page.getByRole('button', { name: 'Login to Practice Account' }).click();

    if (data.shouldPass === 'true') {

      await expect(page).not.toHaveURL(/multiple_element_filter/);

    } else {

      await expect(page.getByText(data.expectedError)).toBeVisible();

    }

  });

}
```



```
    }

    });

}

// csvReader.ts - a ~30 line hand-rolled parser

import * as fs from 'fs';

import * as path from 'path';

export interface TestDataRow { [key: string]: string }

export function readCSV(filePath: string): TestDataRow[] {

    const content = fs.readFileSync(path.resolve(filePath), 'utf-8');

    const [header, ...lines] = content.trim().split("\n");

    const headers = header.split(',').map(h => h.trim());

    return lines.map(line => {

        const values = line.split(',');

        return Object.fromEntries(headers.map((h, i) => [h, values[i]??.trim() ?? ""])) as
TestDataRow;

    });

}
```

Data Source	Reader	When to Use
<code>inline const rows = [...]</code>	none	tiny matrices (< 10 rows)
<code>.json</code>	<code>import data from './x.json'</code>	structured payloads, TS validates shape
<code>.csv</code>	<code>readCSV(path)</code>	non-coders edit in Excel
<code>.yaml</code>	<code>readYAML(path, key?)</code>	human-friendly, comments, nesting



Data Source	Reader	When to Use
<code>.xlsx</code>	<code>readXLSX(path, sheet?)</code>	real business datasets in spreadsheets

Read the file at module top-level (outside `test()`), or Playwright finishes test discovery before your loop runs and registers zero tests. The Faker labs (266-269) generate fresh fake users each run instead of using a static file.

Page Object Model (Labs 270-271)

The Page Object Model (POM) moves selectors and actions out of the spec into a reusable class, so tests read like plain English and locators live in one place.

// Without POM (Lab 270) - selectors scattered inside the spec

```
await page.getByRole('textbox', { name: 'Email' }).fill('user@test.com');
```

```
await page.getByRole('textbox', { name: 'Password' }).fill('secret');
```

```
await page.getByRole('button', { name: 'Login' }).click();
```

// LoginPage.ts - the page object

```
export class LoginPage {
```

```
  constructor(private page: Page) {}
```

```
  async login(user: string, pass: string) {
```

```
    await this.page.getByRole('textbox', { name: 'Email' }).fill(user);
```

```
    await this.page.getByRole('textbox', { name: 'Password' }).fill(pass);
```

```
    await this.page.getByRole('button', { name: 'Login' }).click();
```

```
  }
```

```
}
```

// With POM (Lab 271) - the spec reads cleanly

```
const loginPage = new LoginPage(page);
```

```
await loginPage.login('user@test.com', 'secret');
```



In Simple Terms: a page object is a class that holds one page's locators and actions. Tests call `loginPage.login(...)` instead of repeating selectors, so a UI change means editing one file.

■ Real Project: TTA Bank E2E

The capstone is a real banking app: sign up, transfer funds, then verify the balance dropped.

`Task1.spec.ts` demonstrates TypeScript interfaces for test data (`SignUpData`, `TransferData`), helper functions (`fillSignUpForm`, `transferFunds`, `confirmTransfer`, `verifyDashboardBalance`), accessibility-first locators (`getByRole`, `getByPlaceholder`, `getByText`), and multi-step assertions (heading visible → button visible → balance updated → activity shows a debit).

■ Real Project: TTACart (POM E2E)

`TTACartProject` is a full Page Object Model flow for the TTACart demo: login → inventory → add products → cart → checkout. Credentials come from a `.env` file via `dotenv`.

// three page objects drive the flow

`TTACartLoginPage.ts` // login actions

`TTACartInventoryPage.ts` // product selection, cart, checkout navigation

`TTACartCheckoutPage.ts` // checkout form completion

`npx playwright test TTACartProject/tests/ttacartE2E.spec.ts --project=chromium`

■ Reporting, CI, and Config

The suite runs Playwright's built-in **HTML reporter** plus a hand-written **Custom TTA reporter** (a great example of the pluggable `Reporter` interface), with **Allure** available behind a config switch.

```
reporter: [['html'], ['./utils/CustomTTAR Reporter.ts']],
```

```
use: {
```

```
  trace: 'on', video: 'on', screenshot: 'on',
```

```
  headless: false, viewport: { width: 1920, height: 1080 },
```

```
},
```

```
projects: [{ name: 'chromium', use: { ...devices['Desktop Chrome'] } }],
```



Setting	Value	Why
<code>fullyParallel</code>	<code>true</code>	speed
<code>retries</code>	2 on CI / 0 locally	catch flakes on CI without slowing dev
<code>trace / video / screenshot</code>	<code>on</code>	full debugging artifacts
<code>headless</code>	<code>false</code>	learning is easier when you watch
<code>projects</code>	<code>chromium</code> only	Firefox / WebKit ready to enable

CI ([.github/workflows/playwright.yml](https://github.com/microsoft/playwright/blob/master/.github/workflows/playwright.yml)) runs on every push/PR: checkout → setup Node → `npm ci` → `npx playwright install --with-deps` → `npx playwright test` → upload the report as an artifact (retained 30 days).

In Simple Terms

- One Browser, many Contexts (one per user), many Pages (tabs)
- Prefer `getByRole` and other user-facing locators, XPath last
- `expect(locator)` auto-retries and must be awaited, value `expect` runs once
- Save the session once with `storageState`, reuse it to skip login
- Data-driven testing loops `test()` over rows, the loop lives outside the test
- The Page Object Model keeps locators in one reusable class

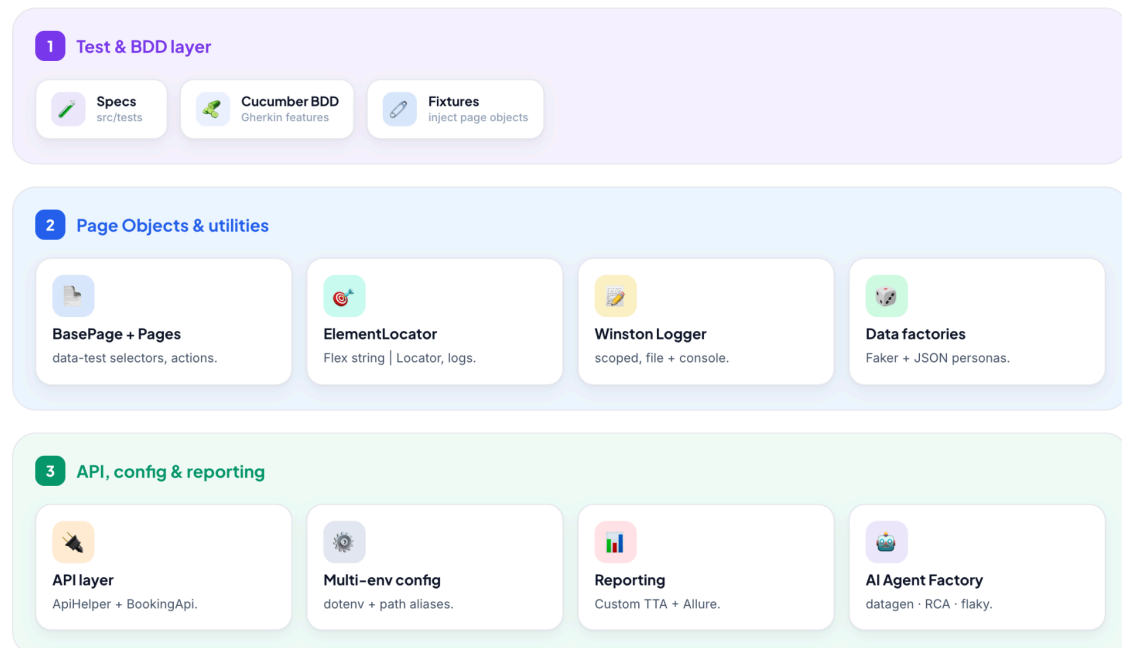
Part 2 - Advanced Playwright Framework

A production-grade Playwright + TypeScript framework: Page Object Model, custom fixtures, an API client layer, data-driven and BDD testing, Winston logging, a custom HTML reporter, and a built-in AI Agent Factory. This is where the fundamentals from Part 1 grow into a real, maintainable framework.



Advanced Framework Architecture

Layered, aliased, opinionated: tests speak in page objects, everything else is plumbing



The Testing Academy · Advance Playwright Framework 1.x

What Makes This a Framework?

A framework is more than a folder of tests. It is an opinionated structure that makes tests **scalable, maintainable, and reusable**. This one bundles:

- Page Object Model (`src/pages/`) with a shared `BasePage`
- Custom fixtures (`src/fixtures/`) that inject page objects and API clients
- An API client layer (`src/api/`) over the RESTful Booker service
- Data-driven testing (JSON, DataTables, Scenario Outlines, Faker factories)
- BDD with Cucumber + Gherkin (`src/cucumber/`) reusing the same page objects
- Winston logging, a custom TTA HTML reporter, and Allure
- Tag-based execution (`@p0`, `@p1`, `@e2e`, `@smoke`, `@lor`)
- An AI Agent Factory (`src/ai/`): an LLM gateway plus data-generation, RCA, and flaky-analysis agents

✅ **Why go from a plain spec to a framework?** Because real suites have hundreds of tests. When a selector changes you want to edit one file, not one hundred. When a new tester joins, the structure should teach them where things go.



■ TypeScript Config and Path Aliases

Path aliases replace ugly `../../../../../` imports with clean `@`-prefixed names. They are defined once in `tsconfig.json` and used everywhere.

```
"paths": {  
  
  "@ai/*": ["/src/ai/*"],  
  
  "@api/*": ["/src/api/*"],  
  
  "@config/*": ["/src/config/*"],  
  
  "@fixtures/*": ["/src/fixtures/*"],  
  
  "@pages/*": ["/src/pages/*"],  
  
  "@testdata/*": ["/src/testdata/*"],  
  
  "@tests/*": ["/src/tests/*"],  
  
  "@utils/*": ["/src/utils/*"]  
}
```

```
// Instead of: import { LoginPage } from '../../pages/LoginPage';
```

```
import { LoginPage } from '@pages/LoginPage';
```

The project rule is simple: **use aliases, never `../../../../../`**. TypeScript strict mode plus `npm run typecheck (tsc --noEmit)` and `npm run lint` are the quality gate on every change.

■ Multi-Environment Configuration

The base URL is chosen at runtime from a `TTA_ENV` variable, so the same suite runs against QA, staging, prod, dev, or the API host without code changes. `dotenv` loads `.env` before any spec.

```
function resolveBaseUrl(): string {  
  
  if (process.env.BASE_URL) return process.env.BASE_URL;  
  
  const env = (process.env.TTA_ENV || 'qa').toLowerCase();  
  
  switch (env) {
```



```
case 'api': return process.env.API_BASE_URL || 'https://restful-booker.herokuapp.com';

case 'dev': return process.env.DEV_BASE_URL || 'http://localhost:3000';

case 'stg': return process.env.STG_BASE_URL ||
'https://stage.thetestingacademy.com';

case 'prod': return process.env.PROD_BASE_URL ||
'https://app.thetestingacademy.com';

case 'qa':

default: return process.env.QA_BASE_URL || 'https://app.thetestingacademy.com';

}

}
```

Run against staging with `TTA_ENV=stg npx playwright test`. Secrets and per-env URLs live in `.env` (never committed), with public demo fallbacks so a fresh clone still runs.

Winston Logging

Instead of `console.log`, the framework uses a Winston logger. `createLogger(scope)` makes a child logger tagged with where the line came from, so every page object logs under its own name. Output goes to the console (colorized) and to `logs/combined.log` for CI artifacts.

```
import winston from 'winston';

const LOG_LEVEL = process.env.LOG_LEVEL ?? 'info';

export const logger = winston.createLogger({

  level: LOG_LEVEL,

  transports: [

    new winston.transports.Console({ /* colorized */ }),

    new winston.transports.File({ filename: 'logs/combined.log' }),

  ],

});

// A scoped child logger - pass the calling class name as the scope
```



```
export function createLogger(scope: string): winston.Logger {  
  
    return logger.child({ scope });  
  
}
```

A log line reads like `2026-06-02 07:40:01 [info] [LoginPage] loginAs standard_user`. The rule is: **use `logger`, not `console.log`**.

Element Utilities (UtilElementLocator)

Rather than calling `locator.click()` directly in page objects, actions go through a small wrapper that logs every action and accepts either a CSS string or a `Locator`. This `Flex` type keeps call sites clean.

```
export type Flex = string | Locator; // a CSS string OR an already-built Locator  
  
export class UtilElementLocator {  
  
    constructor(page: Page, scope = 'UtilElementLocator') {  
  
        this.page = page;  
  
        this.log = createLogger(scope);  
  
    }  
  
    private toLocator(target: Flex): Locator {  
  
        return typeof target === 'string' ? this.page.locator(target) : target;  
  
    }  
  
    async click(target: Flex, timeout = DEFAULT_ACTION_TIMEOUT_MS): Promise<void> {  
  
        const loc = this.toLocator(target);  
  
        this.log.debug(`click ${this.describe(target)}`);  
  
        await loc.click({ timeout });  
  
    }  
  
    async fill(target: Flex, value: string, timeout = DEFAULT_ACTION_TIMEOUT_MS):  
    Promise<void> {
```



```

const loc = this.toLocator(target);

this.log.debug(`fill ${this.describe(target)}`);

await loc.fill(value, { timeout });

}

}

```

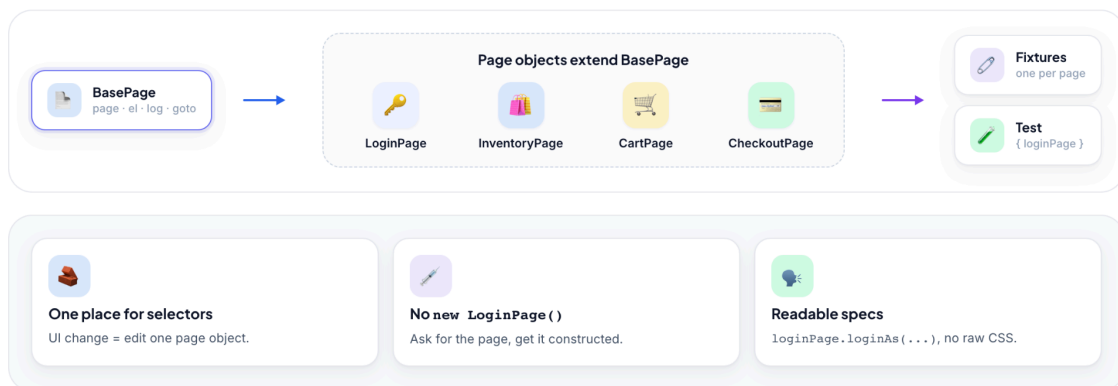
Every click and fill now leaves a log line automatically, which makes failures easy to trace.

The Page Object Model (BasePage)

Every page object extends an abstract **BasePage**. The base deliberately builds **no** locators, it only shares the **page** handle, the element wrapper (**el**), a scoped logger (**log**), and a **goto** helper. Subclasses declare their own **private readonly** locators, selected by stable **data-test** attributes.

Page Objects & Fixtures

BasePage shares the plumbing, fixtures inject ready-built pages into every test



The Testing Academy · Page Object Model + Fixtures

```

export abstract class BasePage {

    protected readonly page: Page;

    protected readonly el: UtilElementLocator;

    protected readonly log: Logger;

    protected constructor(page: Page, scope: string) {

        this.page = page;
    }
}

```



```
this.el = new UtilElementLocator(page, scope);

this.log = createLogger(scope);
}

protected async goto(relativePath: string): Promise<void> {

    await this.page.goto(relativePath);

    await this.page.waitForLoadState('domcontentloaded');

}

}

export class LoginPage extends BasePage {

    static readonly PATH = '/playwright/ttacart/index.html';

    private readonly usernameInput: Locator;

    private readonly passwordInput: Locator;

    private readonly loginButton: Locator;

    constructor(page: Page) {

        super(page, 'LoginPage');

        this.usernameInput = page.locator('[data-test="username"]');

        this.passwordInput = page.locator('[data-test="password"]');

        this.loginButton = page.locator('[data-test="login-button"]');

    }

    async open(): Promise<void> {

        await this.goto(LoginPage.PATH);

    }

    async loginAs(username: string, password: string): Promise<void> {

        this.log.info(`loginAs ${username}`);
```



```
    await this.el.fill(this.usernameInput, username);

    await this.el.fill(this.passwordInput, password);

    await this.el.click(this.loginButton);

  }
}
```

The storefront flow has seven page objects (login → inventory → item detail → cart → checkout step one → step two → complete), each with a `static readonly PATH` and an `assertLoaded()`. A barrel `index.ts` re-exports them all.

In Simple Terms: `BasePage` is the shared parent. Each real page adds its own locators and action methods. Tests speak in page-object verbs like `loginPage.loginAs(...)`, never raw selectors.

■ Custom Fixtures (Page Object Injection)

Instead of writing `new LoginPage(page)` in every spec, the framework extends Playwright's `test` with one fixture per page object. Ask for the page you need and it arrives already constructed against the test's `page`.

```
import { test as base } from '@playwright/test';

import { LoginPage } from '@pages/LoginPage';

import { InventoryPage } from '@pages/InventoryPage';

// ... other page imports

export const test = base.extend<TestFixture>({

  loginPage: async ({ page }, use) => { await use(new LoginPage(page)); },

  inventoryPage: async ({ page }, use) => { await use(new InventoryPage(page)); },

  cartPage: async ({ page }, use) => { await use(new CartPage(page)); },

  // ... one fixture per page object

});

export { expect } from '@playwright/test';
```



```
// A spec now just asks for the pages it needs

import { test, expect } from '@fixtures/test-base';

test('add to cart', async ({ inventoryPage, cartPage }) => {

  await inventoryPage.open();

  await inventoryPage.addToCart('tta-bike-light');

  await cartPage.open();

  expect(await cartPage.rowCount()).toBe(1);

});
```

Fixtures hand over **constructed** (not opened) page objects, because different flows reach a page in different orders, so each spec calls `.open()` itself.

■ Writing Tests: Steps and Per-Step Screenshots

`test.step` groups actions into named, reportable phases. The framework adds a `visualStep` helper that also captures a screenshot at the end of each step and attaches it, so the custom reporter shows one image per step.

```
export async function visualStep(page: Page, title: string, body: () => Promise<void>):
Promise<void> {

  await test.step(title, async () => {

    await body();

    const info = test.info();

    const index = stepCounters.get(info) ?? 0;

    stepCounters.set(info, index + 1);

    await info.attach(`step-${index}-${slugify(title)}`, {

      body: await page.screenshot(),

      contentType: 'image/png',

    });

  });

}
```



```
});  
  
}  
  
await visualStep(page, 'Open the cart', async () => {  
  
    await cartPage.open();  
  
});
```

Test Data Factories (Faker)

Random test data is centralized in a **DataGenerator** static factory built on Faker, so tests stay readable and data never gets hard-coded.

```
import { faker } from '@faker-js/faker';  
  
export class DataGenerator {  
  
    static username(): string {  
  
        return faker.internet.userName();  
  
    }  
  
    static password(length = 12): string {  
  
        return faker.internet.password({ length });  
  
    }  
  
    static credentials(): Credentials {  
  
        return { username: DataGenerator.username(), password: DataGenerator.password() };  
  
    }  
  
    static firstName(): string {  
  
        return faker.person.firstName();  
  
    }  
  
}
```

For the API side, **buildBooking()** in **booking.data.ts** is a Faker payload factory for the RESTful Booker service.



■ Tags, Grep, and CI-Aware Config

Every test carries a priority/type tag, and npm scripts run subsets by tag. The config auto-tunes for CI.

```
// each test is tagged

test.describe.serial('@e2e @P0 Level 3 - Booking lifecycle', () => { /* ... */ });

// package.json scripts filter by tag

"test:p0": "playwright test --grep @p0",

"test:e2e": "playwright test --grep @e2e",

"test:bdd:smoke": "cucumber-js --tags @smoke",

const isCI = !!process.env.CI;

export default defineConfig({

  fullyParallel: true,

  forbidOnly: isCI,

  retries: isCI ? 2 : 0,

  workers: isCI ? 4 : undefined,

});
```

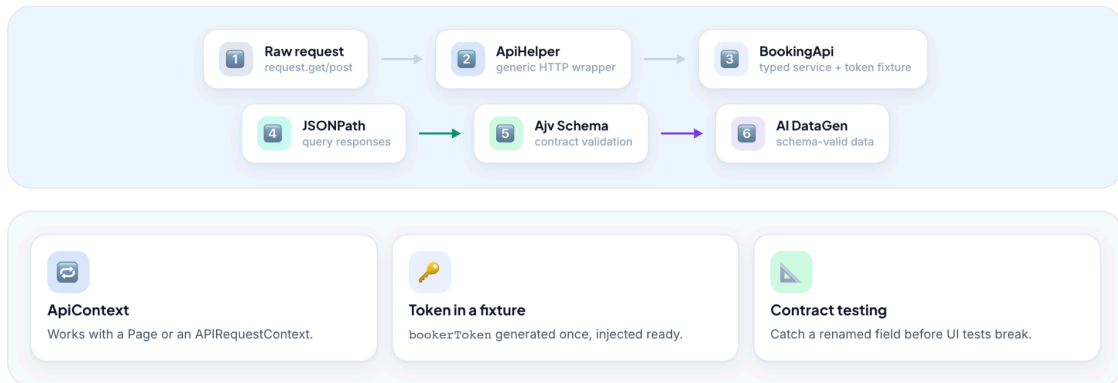
Tags use lowercase (`@p0`) by convention, because the `--grep @p0` scripts are case-sensitive and would silently miss `@P0`.

■ API Testing: Raw to Service to Fixture

The framework teaches API testing as a progression of six layers under `src/tests/apiTests/`, each more reusable than the last. The whole point is to show **why** a framework abstracts raw calls into a service class.

API Testing Progression

Six layers from raw calls to AI-generated data, each more reusable than the last



The Testing Academy · API Testing Layers (RESTful Booker)

Layer	Folder	What it shows
1	01_restfulbooker_raw	raw request fixture CRUD, no helpers
2	02_restfulbooker_apiHelper	wrap calls in ApiHelper
3	03_restfulbooker_fixture_e2e	BookingApi service + token-in-fixture full lifecycle
4	04_jsonpath_plus	query responses with JSONPath
5	05_ajv_schema	validate responses against a JSON Schema
6	06_ai_datagen	AI-generated, schema-valid test data

ApiHelper is a generic HTTP wrapper that accepts either a **Page** or an **APIRequestContext**.

```
export type ApiContext = Page | APIRequestContext;
```

```
export type HttpMethod = 'GET' | 'POST' | 'PUT' | 'DELETE' | 'PATCH';
```

```
export class ApiHelper {
```

```
  async callApi(options: APIRequestOptions): Promise<APIResponse> {
```



```
const { url, method, headers, data, params, timeout } = options;

const request = this.getRequest();

const fullUrl = this.buildUrl(url, params);

switch (method) {

  case 'GET': return request.get(fullUrl, { headers, timeout });

  case 'POST': return request.post(fullUrl, { headers, data, timeout });

  case 'PUT': return request.put(fullUrl, { headers, data, timeout });

  case 'DELETE': return request.delete(fullUrl, { headers, timeout });

  case 'PATCH': return request.patch(fullUrl, { headers, data, timeout });

}

}

}
```

BookingApi is a typed service object built on top of **ApiHelper**, with interfaces for the payload and response.

```
export interface Booking {

  firstname: string;

  lastname: string;

  totalprice: number;

  depositpaid: boolean;

  bookingdates: { checkin: string; checkout: string };

  additionalneeds?: string;

}

export class BookingApi {

  constructor(context: ApiContext, baseUrl = 'https://restful-booker.herokuapp.com') {
```



```
this.apiHelper = new ApiHelper(context);

this.baseUrl = baseUrl;

}

async auth(username = 'admin', password = 'password123'): Promise<string> {

    const response = await this.apiHelper.post(`${this.baseUrl}/auth`, { username,
password });

    const body = await this.apiHelper.parseJsonResponse<{ token?: string }>(response);

    return body.token!;

}

}
```

The auth token is generated inside a **fixture**, so tests receive it ready to use.

```
export const test = base.extend<BookerFixtures>({

    bookingApi: async ({ request }, use) => { await use(new BookingApi(request)); },

    bookerToken: async ({ bookingApi }, use) => {

        const token = await bookingApi.auth();

        await use(token);

    },

});

// A test just asks for the service and the token

test('update the booking', async ({ bookingApi, bookerToken }) => {

    const updated = await bookingApi.updateBooking(bookingId, payload, bookerToken);

    expect(updated.lastname).toBe('Updated');

});
```



JSONPath and JSON Schema Validation

Once you have a response, JSONPath ([jsonpath-plus](#)) queries deep into it, and Ajv validates the shape against a JSON Schema (contract testing).

// schemaValidator.ts - Ajv Draft-07 validation

```
export function validateSchema(schema: object, data: unknown): { valid: boolean; errors?: string[] } {

  const validate = ajv.compile(schema);

  const valid = validate(data);

  return { valid, errors: validate.errors?.map(e => `${e.instancePath} ${e.message}`) };

}
```

Schema validation catches a backend that quietly renames or drops a field, before it breaks a hundred UI tests.

Data-Driven Testing (JSON, DataTable, Scenario Outline)

Beyond Faker, the framework drives tests from external data in three Cucumber styles: JSON personas, Gherkin DataTables, and Scenario Outlines with Examples.

Scenario Outline: <username> logging in ends on the <outcome>

When I log in as "<username>" with password "<password>"

Then I should see the "<outcome>"

Examples: valid users

username	password	outcome
standard_user	tta_secret	products
problem_user	tta_secret	products
performance_glitch_user	tta_secret	products

Examples: rejected attempts

username	password	outcome
locked_out_user	tta_secret	error



```
| standard_user | wrong_password | error |
```

Each Examples row becomes its own test, so one scenario covers the entire login matrix.

BDD with Cucumber

The BDD layer writes tests in plain-English Gherkin and reuses the **same** page objects as the Playwright tests. A **CustomWorld** holds the browser, page, and all page objects, and hooks manage the browser lifecycle.

@level1 @login

Feature: TTA Cart Login

Background:

Given I am on the TTACart Login Page

@smoke @P0

Scenario: A standard user can log in

When I log in as "standard_user" with password "tta_secret"

Then I should land on the products page

// Step definitions call the SAME page objects

```
import { Given, When, Then } from '@cucumber/cucumber';
```

```
import { CustomWorld } from '../support/world';
```

```
When('I log in as {string} with password {string}',
```

```
  async function (this: CustomWorld, username: string, password: string) {
```

```
    await this.loginPage.loginAs(username, password);
```

```
  });
```

```
Then('I should land on the products page', async function (this: CustomWorld) {
```

```
  await this.inventoryPage.assertLoaded();
```

```
});
```



The curriculum is levelled: level-00 (installation smoke), level-01 (basic login), level-02 (data-driven with outlines, DataTables, and external JSON personas). A custom formatter bridges Cucumber runs into the same TTA HTML report.

Custom TTA HTML Reporter and Allure

The framework ships a hand-written reporter that implements Playwright's **Reporter** interface, producing a branded, self-contained HTML report with per-step screenshots, videos, traces, and filters. Cleverly, the same reporter is reused for Cucumber runs through a formatter that calls `renderExternalRun(...)`.

```
reporter: [  
  ['./src/utls/CustomReporter.ts'],      // custom TTA report  
  
  ['html', { outputFolder: 'playwright-report' }],  
  
  ['json', { outputFile: 'test-results/results.json' }],  
  
  ['allure-playwright', { resultsDir: 'allure-results' }],  
  
  ['list'],  
  
],  
  
use: {  
  screenshot: 'only-on-failure',  
  
  video: 'on',  
  
  trace: 'on-first-retry',  
  
},
```

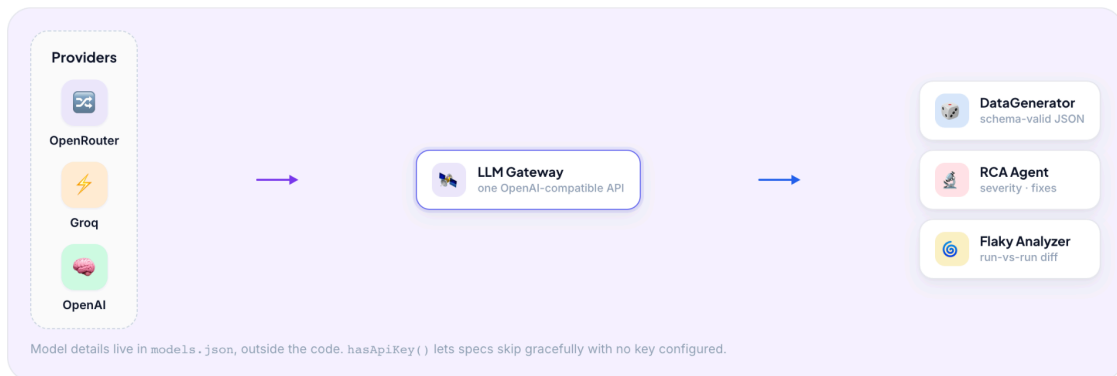
The custom reporter also feeds the AI tabs (AI Data, AI Verdict, AI Flaky) described next.

AI Agent Factory

The framework's standout feature is a built-in AI layer. A single **LLM gateway** talks to OpenRouter, Groq, or OpenAI (they share the same OpenAI-compatible API), selected by environment variables. Model details live in `models.json`, outside the code.

The AI Agent Factory

One LLM gateway, many providers, three agents feeding the test report



Plan → Generate → Heal

The Testing Academy · AI Agent Factory

```
export function llmGateway(overrides: ProviderOverrides = {}): LLMGateway {
  const resolved = resolveProvider(overrides);

  const endpoint = `${resolved.baseUrl}/chat/completions`;

  async function chat(opts: ChatOptions): Promise<ChatResult> {
    const body = { model: opts.model ?? resolved.model, messages: opts.messages,
    temperature: 0 };

    if (opts.jsonMode !== false) body.response_format = { type: 'json_object' };

    // native fetch, no SDK
  }

  return { chat };
}
```

Three agents build on the gateway:

- **Data generator** (`customDataGenerator.ts`): a JSON structure + a prompt → LLM → Ajv-validated → a unique `testdata.json`



- **RCA agent** (`rcaAgent.ts`): a failed test → { `severity`, `priority`, `rootCause`, `fixes` }, shown in the report's AI Verdict tab
- **Flaky analyzer** (`flakyAnalyzer.ts`): diffs two runs to find status-flips, with an optional LLM summary

```
export async function analyzeFailure(failure: FailureInput): Promise<RcaVerdict> {

  const gw = llmGateway();

  const res = await gw.chat({

    messages: [

      { role: 'system', content: SYSTEM }, // "return ONLY JSON: severity, priority, rootCause, fixes"

      { role: 'user', content: `FAILED TEST: ${failure.title}\nERROR:\n${failure.error}` },

    ],

    jsonMode: true,

  });

  const v = extractJson<Partial<RcaVerdict>>(res.content);

  return { severity: v.severity ?? 'Unknown', priority: v.priority ?? 'Unknown', /* ... */ };

}
```

A `hasApiKey()` check lets these specs skip gracefully when no LLM key is configured, so the suite still runs on machines without AI access.

■ AI Test Agents (Plan, Generate, Heal)

Separate from the runtime AI, the repo ships framework-aware IDE agents under `.github/agents/`: a **planner** (URL or feature → a test plan in `specs/`), a **generator** (plan → a POM-based spec), and a **healer** (a failing test → a layer-correct fix). Each has an MCP variant (driven by the Playwright MCP server) and a token-cheap CLI variant. They know the framework's conventions: page objects, fixtures, tags, and API quirks.

■ Quality Gates and Rules



Every change under `src/tests/**` must pass `npm run typecheck` (`tsc --noEmit`) and `npm run lint` (ESLint flat config + Prettier) before it is considered done. The rules are written once in `rules/` and mirrored into rule files for nine AI assistants (Claude, Copilot, Cursor, Windsurf, Augment, and more), so every tool enforces the same standards.

Core rules:

- Page objects only, no raw selectors in specs
- Use `logger`, not `console.log`
- Use path aliases, no `../../../../`
- Every test carries a tag
- No `test.only` / `test.skip` without a ticket

CI/CD with Playwright

CI (`.github/workflows/playwright.yml`) runs the suite on every push and PR, with the config auto-tuning retries and workers for CI. Reports upload as artifacts. Environment variables select the target env and the LLM provider.

Note: the repo includes a `Dockerfile`, but it is currently empty (a scaffold), so containerized runs are a natural next exercise rather than a finished feature.

In Simple Terms

- A framework makes tests scalable, maintainable, and reusable
 - Path aliases, multi-env config, and Winston logging are the foundation
 - `BasePage` + page objects + fixtures keep selectors in one place and inject them cleanly
 - The API layer grows from raw calls to a typed service with a token fixture
 - Cucumber reuses the same page objects for plain-English BDD
 - A custom reporter and Allure make results readable, and the AI Agent Factory adds data generation, root-cause analysis, and flaky detection
-



The Practice Playground at a Glance

One hand-drawn map of the whole learning journey, from your first locator to a full framework.

TheTestingAcademy